DESENVOLVIMENTO DE SISTEMAS WEB I

Spring MVC e Vue.js: por que e como começar

Do que já construímos até o passo a passo para criar os dois projetos do Sistema de Vendas

Aula introdutória — as telas e a lógica do sistema serão construídas em sala

PONTO DE PARTIDA

Objetivos da aula

1. **Entender por que trocamos Servlet e JavaScript puro por Spring e Vue**

   Partindo do que já construímos: onde doeu e o que cada framework resolve.

2. **Distinguir Spring, Spring MVC e Spring Boot**

   Três nomes que aparecem juntos e que costumam confundir.

3. **Criar o back-end (vendas-api) e o front-end (vendas-web)**

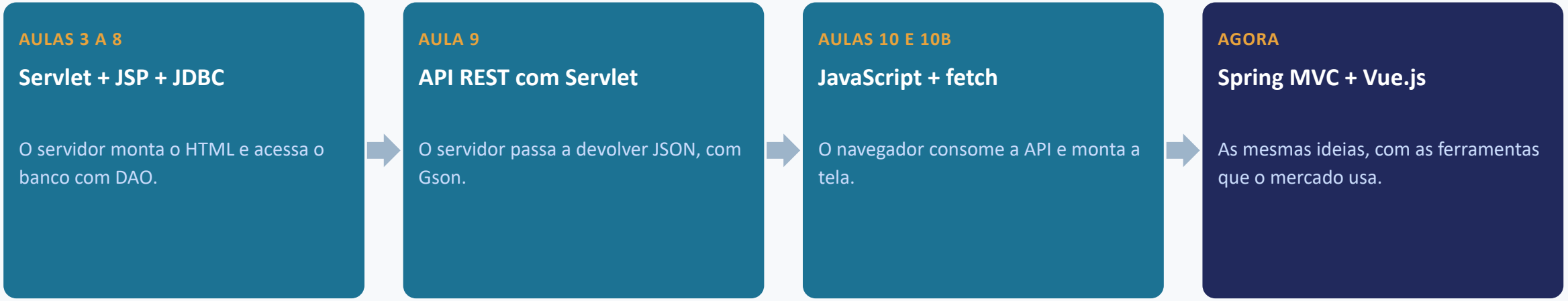
   Dois projetos independentes, gerados com o Spring Initializr e o create-vue.

4. **Fazê-los conversar**

   Um teste de ponta a ponta: o Vue mostra na tela o que a API Spring devolveu.

DE ONDE VIEMOS

A jornada até aqui



Nada do que aprendemos foi jogado fora: MVC, REST, JSON, DAO, fetch e DOM continuam valendo. Os frameworks automatizam o trabalho repetitivo, e só entendemos o que eles escondem porque já fizemos tudo isso à mão.

O QUE SENTIMOS NA PELE

Onde o caminho manual começou a pesar

NO BACK-END (SERVLET + JDBC)

- Ler o id do pathInfo e tratar erro de conversão à mão (o extrairId)
- Escrever cada status HTTP (201, 204, 400, 404) método a método
- Converter JSON com Gson em cada resposta e em cada leitura
- DAO repetitivo: Connection, PreparedStatement, ResultSet, try-with-resources
- Instalar o Tomcat, gerar o .war e lidar com o CONTEXT_PATH

NO FRONT-END (JAVASCRIPT PURO)

- Atualizar a tela na mão: getElementById, innerHTML, classList
- Lembrar de usar escapeHtml sempre que se usa innerHTML
- Copiar HTML parecido (Novo e Editar) e manter os dois sincronizados
- Recalcular lista e total a cada mudança (o painel de Venda vem aí)
- Scripts na ordem certa, páginas separadas, estado espalhado

O que fazíamos x o que o Spring oferece

O que fazíamos	O que o Spring oferece
@WebServlet + doGet, doPost, doPut, doDelete	@RestController com @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
Extrair o id do pathInfo e converter	@PathVariable na rota /api/produtos/{id}
Gson: toJson e fromJson em cada método	Conversão automática de e para JSON (@RequestBody e o próprio retorno)
Escrever o status HTTP à mão	ResponseEntity e tratamento centralizado de erros
ContatoDAO com JDBC puro	Repository do Spring Data JPA: o CRUD básico já vem pronto
ConnectionFactory (URL, usuário, senha)	Datasource configurado no application.properties
if para validar nome em branco	Anotações de validação: @NotBlank, @Positive
Instalar Tomcat e gerar .war	Tomcat embutido: o projeto roda sozinho, sem instalar servidor

O que fazíamos x o que o Vue oferece

O que fazíamos	O que o Vue oferece
getElementById + textContent ou innerHTML	Template com {{ }} e v-for: a tela acompanha os dados sozinha
escapeHtml em todo innerHTML	A interpolação {{ }} já escapa o texto (v-html é a exceção)
classList.add e remove de d-none	v-if e v-show: mostrar ou esconder por condição
FormData e ler cada input	v-model: o campo e a variável ficam ligados
Recalcular o total do resumo manualmente	computed: valor derivado que se atualiza sozinho
Copiar o HTML de Novo e de Editar	Componentes .vue reutilizáveis
Trocar de tela com links e vários scripts	Vue Router: o menu troca o painel na mesma página
Script solto + DOMContentLoaded	Ciclo de vida do componente (onMounted)

VOCABULÁRIO

Spring, Spring MVC e Spring Boot: quem é quem

SPRING FRAMEWORK

A base de tudo. Seu coração é o contêiner que cria os objetos da aplicação (beans) e os entrega a quem precisa: a injeção de dependência.

SPRING MVC

O módulo web do Spring. Um único servlet (o DispatcherServlet) recebe todas as requisições e as distribui para os métodos dos controllers. É o papel que escrevíamos à mão em cada @WebServlet.

SPRING BOOT

O jeito pronto de usar o Spring: starters de dependências, configuração automática, Tomcat embutido e o Spring Initializr para gerar o projeto. Roda com um comando.

Na prática: "criar um projeto Spring MVC" é gerar um projeto Spring Boot com a dependência Spring Web.

O MVC continua: mudou onde cada peça mora

Papel	Antes (Servlet + JSP)	Agora (Spring MVC + Vue)
View	JSP gera o HTML no servidor	Vue desenha a tela no navegador a partir do JSON
Controller	Servlet (doGet, doPost...)	@RestController do Spring MVC
Model	Contato + ContatoDAO (JDBC)	Entity + Repository (JPA), com Service para as regras

A separação de responsabilidades é a mesma que aprendemos na Aula 3. A grande mudança: o servidor deixou de entregar páginas prontas e passou a entregar apenas dados (JSON). A View ficou no navegador.

VISÃO GERAL

A arquitetura do Sistema de Vendas



Dois projetos independentes, duas portas, dois terminais. O front-end não conhece o banco: ele só conversa com a API.

Injeção de dependência: o Spring cria os objetos

Até aqui, quem precisava de um DAO fazia new ContatoDAO(). No Spring, a classe apenas declara do que precisa, e o Spring entrega:

```
@RestController
public class ProdutoController {

    private final ProdutoRepository repository;

    public ProdutoController(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

(exemplo ilustrativo: as classes serão criadas em sala)

- Nenhum new: o Spring, ao iniciar, instancia as classes marcadas (@RestController, @Service, @Repository) e as chama de beans
- O que a classe precisa chega pelo construtor
- Mesmo espírito do ConnectionFactory: um lugar só decide como o objeto é criado
- Trocar ou testar uma peça fica mais simples

CONCEITO DO VUE

Reatividade: os dados mandam, a tela acompanha

```
<script setup>
import { ref, computed } from 'vue'

const itens = ref([{ nome: 'Caneta', preco: 2.5, qtd: 4 }])

const total = computed(() => {
  let soma = 0
  for (const item of itens.value) {
    soma += item.preco * item.qtd
  }
  return soma
})
</script>

<template>
  <p v-for="i in itens" :key="i.nome">{{ i.nome }}</p>
  <p>Total: {{ total }}</p>
</template>
```

- ref() cria uma variável reativa: ao mudar itens.value, o Vue atualiza a tela
- computed() guarda um valor derivado: o total se recalcula sozinho quando a lista muda
- v-for desenha uma linha por item; {{ }} escreve o valor (já escapado)
- Compare com o painel de Venda manual: nenhuma linha de getElementById ou textContent

Prévia do conceito. Não é o código do projeto: ele será escrito em sala.

A1

PARTE A · BACK-END · SPRING MVC

Antes de começar: o que conferir

Item	Como conferir	Observação
JDK 25	`java -version`	Versão do curso. O Spring Boot 4 exige Java 17 ou superior e funciona com o 25
IDE	`IntelliJ IDEA Ultimate ou VS Code`	VS Code com Extension Pack for Java e Spring Boot Extension Pack
PostgreSQL no Docker	`docker ps`	O contêiner pg-agenda (ou o seu) deve aparecer ligado na porta 5432
Cliente de API	`Thunder Client ou Postman`	Opcional: o navegador já testa os GET
Maven	`nada a instalar`	O projeto gerado traz o Maven Wrapper (mvnw)

A2

PARTE A · BACK-END · SPRING MVC

Gerar o projeto no Spring Initializr

Acesse start.spring.io e preencha o formulário:

Campo	Valor	Observação
Project	`Maven`	O mesmo sistema de build que já usamos
Language	`Java`	
Spring Boot	`4.1.x (estável mais recente)`	Escolha uma versão sem SNAPSHOT, RC ou M
Group / Artifact / Name	`br.edu.faculdade / vendas-api / vendas-api`	O nome vira o da classe principal (VendasApiApplication)
Package name	`br.edu.faculdade.vendas`	Edite este campo: o padrão é montado com o artifact
Packaging	`Jar`	O Tomcat vem embutido, não precisa de .war
Java	`25`	A versão do nosso JDK

A3

PARTE A · BACK-END · SPRING MVC

Adicionar as dependências (botão Add Dependencies)

Dependência no Initializr	Para que serve	O que fazíamos antes
Spring Web	Spring MVC, conversão para JSON e Tomcat embutido	Servlet API + Gson + Tomcat instalado
Spring Data JPA	Repositories e mapeamento de classes para tabelas (Hibernate)	ContatoDAO com JDBC
PostgreSQL Driver	Driver JDBC do PostgreSQL	A dependência postgresql do pom.xml (Aula 8)
Validation	Anotações de validação (@NotBlank, @Positive)	ifs de validação no Servlet
Spring Boot DevTools (opcional)	Reinicia a aplicação ao salvar o código	Gerar o .war e republicar a cada mudança

Depois clique em Generate: o Initializr baixa um .zip. Descompacte na pasta do projeto, por exemplo sistema-vendas/vendas-api.

A4

PARTE A · BACK-END · SPRING MVC

Abrir na IDE e conhecer a estrutura

```
vendas-api/
  pom.xml
  mvnw  mvnw.cmd  .mvn/
  src/main/java/br/edu/faculdade/vendas/
    VendasApiApplication.java
  src/main/resources/
    application.properties
  src/test/java/br/edu/faculdade/vendas/
    VendasApiApplicationTests.java

// pacotes que criaremos em sala:
//   controller, service, repository, model
```

O QUE CADA PARTE FAZ

`pom.xml`

As dependências escolhidas viram starters aqui. É o mesmo papel do pom.xml das aulas anteriores.

`mvnw / mvnw.cmd`

Maven Wrapper: roda o Maven sem instalá-lo.

`VendasApiApplication.java`

Tem o @SpringBootApplication e o main. Controllers abaixo deste pacote são encontrados automaticamente.

`application.properties`

Toda a configuração: banco, porta, logs. Substitui o web.xml e o ConnectionFactory.

A5

PARTE A · BACK-END · SPRING MVC

Criar o banco vendas no PostgreSQL do Docker

```
docker ps

docker exec -it pg-agenda psql -U postgres -c "CREATE DATABASE
vendas;"
```

O servidor é um só, com um banco por projeto: agenda_telefonica continua intacto.

O QUE CADA PARTE FAZ

`docker ps`

Confirma que o contêiner está ligado. Se o nome for outro, use o nome que aparecer.

`docker exec ... psql`

Executa o psql dentro do contêiner e cria o banco. O servidor PostgreSQL é o mesmo da Agenda; o banco é novo.

`Pelo pgAdmin`

Botão direito em Databases, Create, Database..., nome vendas. O resultado é o mesmo.

`Conferir`

O banco vendas deve aparecer na lista de Databases.

A6

PARTE A · BACK-END · SPRING MVC

Configurar o application.properties

```
spring.application.name=vendas-api
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/vendas
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

Sem este arquivo preenchido a aplicação não sobe: o Spring Data JPA exige um banco acessível na inicialização.

O QUE CADA PARTE FAZ

`spring.datasource.*`

A mesma URL, usuário e senha que ficavam no ConnectionFactory (DatabaseConnect). Ajuste a porta se a sua for outra (ex.: 5433).

`ddl-auto=validate`

O Hibernate só confere se as tabelas existem. As tabelas continuam nascendo da nossa modelagem (Aula 7).

`show-sql=true`

Imprime no console o SQL gerado: dá para comparar com o que escrevíamos à mão no DAO.

`server.port`

A porta do Tomcat embutido.

A7

PARTE A · BACK-END · SPRING MVC

Primeiro endpoint: um teste de vida

Crie a classe PingController no pacote br.edu.faculdade.vendas.controller:

```
@RestController
@RequestMapping("/api")
public class PingController {

    @GetMapping("/ping")
    public Map<String, String> ping() {
        return Map.of("status", "ok");
    }
}
```

Imports: Map (java.util) e as anotações de org.springframework.web.bind.annotation. A IDE sugere sozinha.

O QUE CADA PARTE FAZ

`@RestController`

A classe é um controller e o retorno de cada método vira o corpo da resposta (JSON). Substitui escrever no response do Servlet.

`@RequestMapping("/api")`

Prefixo comum das rotas, como o /api/* do nosso @WebServlet.

`@GetMapping("/ping")`

Responde a GET /api/ping. Era o doGet.

`Map.of(...)`

O Spring converte o Map em {"status":"ok"} sozinho. Nada de Gson.

A8

PARTE A · BACK-END · SPRING MVC

Rodar e testar a API

```
# Windows
mvnw.cmd spring-boot:run

# Linux / macOS
./mvnw spring-boot:run
```

RESULTADO ESPERADO

Console: Tomcat started on port 8080 e Started VendasApiApplication.
Navegador: abra http://localhost:8080/api/ping e veja {"status":"ok"}.

Erro comum	Causa provável	O que fazer
Port 8080 was already in use	Outro programa (o Tomcat antigo?) usa a porta	Pare o outro ou mude server.port
Failed to obtain JDBC Connection	Contêiner parado ou URL, porta, senha ou banco errados	docker ps e confira o application.properties
404 em /api/ping	Controller fora do pacote da classe principal	Mova para um subpacote de br.edu.faculdade.vendas
Erro de versão do Java	O JDK da IDE ou do terminal não é o 25	java -version e a configuração da IDE

B1

PARTE B · FRONT-END · VUE.JS

Antes de começar: o que conferir

Item	Como conferir	Observação
Node.js LTS	`node -v`	Versão LTS atual (22 ou superior). O Node roda as ferramentas do Vue, não o sistema em si
npm	`npm -v`	Vem com o Node. É o "Maven do JavaScript": baixa e organiza as dependências
VS Code	`extensão Vue - Official`	Dá autocompletar e destaque nos arquivos .vue. Não use a antiga Vetur
Navegador	`Chrome ou Edge com DevTools (F12)`	Já usamos nas aulas de fetch

B2

PARTE B · FRONT-END · VUE.JS

Criar o projeto com o create-vue

```
npm create vue@latest
```

Pergunta do assistente	Resposta sugerida	Por quê
Project name	`vendas-web`	Nome da pasta e do projeto
Add TypeScript?	`No`	Seguimos em JavaScript, como no restante do curso
Add JSX Support?	`No`	Usaremos os templates .vue
Add Vue Router?	`Yes`	Login e o menu que troca os painéis
Add Pinia?	`No (por enquanto)`	Só entra se precisarmos de estado compartilhado
Add Vitest / End-to-End Testing?	`No`	Testes automatizados ficam fora do escopo
Add ESLint? / Add Prettier?	`Yes / Yes`	Avisa erros no editor e formata o código

As perguntas podem variar um pouco conforme a versão do create-vue: para o que não estiver na tabela, responda No.

Instalar as dependências e subir o servidor

```
cd vendas-web
npm install
npm run dev
```

Para produção existe o npm run build, que gera a pasta dist/. Não precisaremos dele agora.

O QUE CADA PARTE FAZ

`cd vendas-web`

Entra na pasta que o assistente criou.

`npm install`

Baixa as dependências listadas no package.json para a pasta node_modules. É o equivalente ao Maven baixando o que está no pom.xml.

`npm run dev`

Sobe o servidor de desenvolvimento do Vite, com recarga automática ao salvar.

`http://localhost:5173`

Abra no navegador: a página inicial do Vue deve aparecer.

Conhecer a estrutura do projeto

```
vendas-web/
  index.html
  package.json
  vite.config.js
  src/
    main.js
    App.vue
    router/index.js
    views/        (telas inteiras)
    components/   (peças reutilizáveis)
    assets/
```

A estrutura exata pode variar um pouco com a versão do create-vue.

O QUE CADA PARTE FAZ

`index.html`

A única página HTML. Tem um onde o Vue desenha tudo.

`main.js`

Cria o aplicativo Vue, instala o router e monta no #app.

`App.vue`

Componente raiz: o que aparece em todas as telas.

`router/index.js`

Associa endereços a telas (views).

`package.json / vite.config.js`

Dependências e scripts; e a configuração do servidor de desenvolvimento.

B5

PARTE B · FRONT-END · VUE.JS

Anatomia de um componente .vue

```
<script setup>
import { ref, onMounted } from 'vue'

const status = ref('carregando...')

onMounted(async () => {
  const response = await fetch('/api/ping')
  const dados = await response.json()
  status.value = dados.status
})
</script>

<template>
  <h1 class="m-4">API: {{ status }}</h1>
</template>
```

Este será o nosso teste de ponta a ponta (passo B8). Um arquivo .vue ainda pode ter um bloco <style> para o CSS do componente.

O QUE CADA PARTE FAZ

`<script setup>`

A lógica do componente, em JavaScript: variáveis, funções, chamadas à API.

`ref('carregando...')`

Variável reativa. No script se acessa por .value; no template, direto.

`onMounted(...)`

Roda quando o componente aparece na tela. O mesmo papel do DOMContentLoaded.

`<template>`

O HTML do componente. {{ status }} mostra o valor e se atualiza quando ele muda.

B6

PARTE B · FRONT-END · VUE.JS

Bootstrap e limpeza do template inicial

1. Bootstrap pelo CDN, no <head> do index.html (como já fazemos):

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
```

2. Deixar o projeto "limpo" para começar:

- Apague os exemplos do template (HelloWorld, TheWelcome, ícones) e o CSS padrão importado em main.js
- Deixe o App.vue só com o <RouterView />, que mostrará a tela da rota atual
- Mantenha uma view simples (HomeView.vue) para os testes
- Rode npm run dev de novo e confira se a página abre sem erros no Console (F12)

Alternativa mais "profissional": instalar o Bootstrap com npm install bootstrap e importar o CSS no main.js. Para começar, o CDN é suficiente.

B7

PARTE B · FRONT-END · VUE.JS

Ligar o front ao back-end: proxy ou CORS

O Vue roda na porta 5173 e a API na 8080: origens diferentes. Há dois caminhos para o navegador aceitar a conversa:

OPÇÃO 1 (RECOMENDADA): PROXY DO VITE

```
// vite.config.js, dentro do defineConfig:
server: {
  proxy: {
    '/api': 'http://localhost:8080'
  }
}
```

O Vue chama /api/ping na própria porta 5173 e o Vite repassa à API. Para o navegador é a mesma origem, então não há CORS. Vale só no desenvolvimento.

OPÇÃO 2: CORS NO SPRING

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
  @Override
  public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
        .allowedOrigins("http://localhost:5173");
  }
}
```

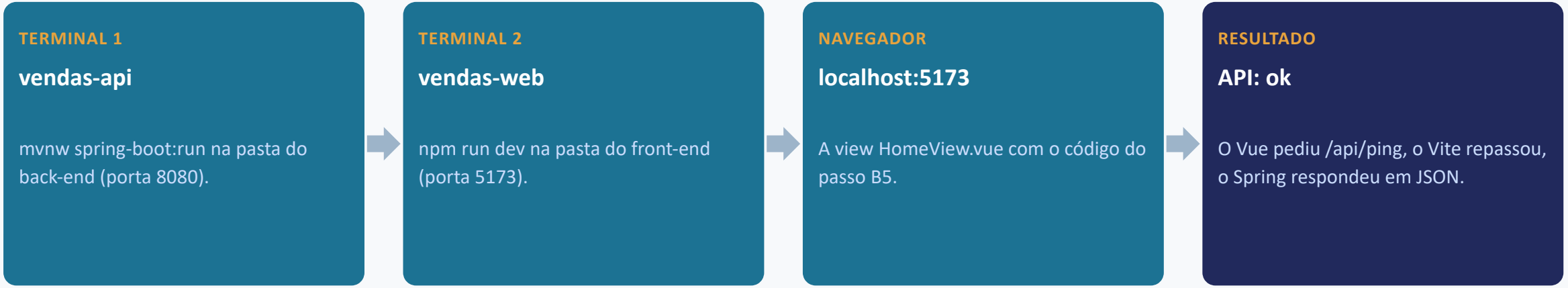
O servidor declara quais origens aceita. É o caminho quando o Vue chama o endereço completo da API (http://localhost:8080) em vez de usar o proxy.

Vamos começar pelo proxy. O CORS volta quando falarmos de publicação, com o front e a API em endereços diferentes.

B8

PARTE B · FRONT-END · VUE.JS

Teste de ponta a ponta



- Se aparecer "API: carregando..." para sempre, abra o Console (F12): erro 404 indica o proxy ou o controller; erro de rede indica a API parada
- Se aparecer "API: ok", o caminho completo funciona: navegador, Vite, Spring MVC e de volta
- A partir daqui, cada tela do Sistema de Vendas é uma variação desta mesma conversa

CHECKLIST FINAL

Tudo pronto para começar o Sistema de Vendas

BACK-END · vendas-api

- [ ] java -version mostra 25
- [ ] Banco vendas criado no PostgreSQL do Docker
- [ ] mvnw spring-boot:run sobe sem erros
- [ ] /api/ping devolve {"status":"ok"}

FRONT-END · vendas-web

- [ ] node -v e npm -v respondem
- [ ] npm run dev abre em localhost:5173
- [ ] Bootstrap carregando no index.html
- [ ] A tela mostra "API: ok" via proxy

Em sala: criar as telas e a lógica, etapa por etapa, seguindo o roteiro do Sistema de Vendas.